

Guía complementaria del proyecto

Física, flutter, programación y lectura de gráficas

Estado del proyecto: prototipo computacional verificado con pruebas numéricas, **no validado experimentalmente**. No autoriza fabricar ni lanzar un cohete real. Cubre únicamente el ascenso libre, desde el abandono del riel hasta el apogeo.

Capítulos incluidos en este PDF: 01 · Física desde cero — 02 · Flutter desde cero —
03 · Programación desde cero — 05 · Todas las gráficas, explicadas desde sus ejes

Repositorio completo, capítulos 00, 04, 06, 07 y 08, código y resultados de referencia:
github.com/fabricioarce/rocket-structural-sim

Documento generado a partir de docs/01, docs/02, docs/03 y docs/05 del repositorio (fuente editable en docs/pdf/).

Contenido

01 · Física desde cero: del vuelo al aire que ve la aleta	3
1. Unidades y magnitudes	3
2. Posición, velocidad y aceleración	3
3. Fuerzas y motor	3
4. Velocidad respecto al aire	4
5. Atmósfera y presión dinámica	4
6. Ángulo de ataque, Mach y estabilidad	4
7. Qué entrega RocketPy y qué añade este proyecto	4
8. Velocidad del sonido y por qué importa para el flutter	5
02 · Flutter desde cero: aeroelasticidad de una aleta	6
1. Aeroelasticidad: por qué una aleta puede vibrar	6
2. Vibración, resonancia y flutter no son sinónimos	6
3. Rigidez,	
	G
y torsión	6
4. Geometría de la aleta	7
5. Fórmula de Bennett (2023), término por término	7
6. Ratio, margen 1.25 y Max-Q	8
7. Espesor requerido	9
8. Barrido algebraico frente a resimulación	9
9. Escala uniforme y relación con Max-Q	9
10. Qué no dice el modelo	10
03 · Programación desde cero: convertir una ecuación en un estudio	11
1. Un programa y un módulo	11
2. Variables y unidades	11
3. Funciones y objetos	11
4. Invertir una función	11
5. Listas, bucles y barridos	11
6. Diccionarios y resultados	12
7. Decisiones y pruebas	12
8. Flujo completo	12
05 · Las siete gráficas, explicadas desde sus ejes	13
Figura 01 · 01-flight.png	13
Figura 02 · 02-flutter-history.png	14
Figura 03 · 03-altitude.png	15
Figura 04 · 04-thickness-sweep.png	15
Figura 05 · 05-geometry-map.png	16
Figura 06 · 06-design-space.png	16
Figura 07 · 07-drag-sensitivity.png	17
Cómo escribir un pie de figura	17

01 · Física desde cero: del vuelo al aire que ve la aleta

Siguiente: flutter

Este capítulo construye el vocabulario mínimo para leer una trayectoria. No se necesita aceptar una cifra porque «parece razonable»: primero se revisan unidades, intervalo y definición.

1. Unidades y magnitudes

Magnitud	Símbolo	Unidad
Tiempo	t	s
Posición	x, y, z	m
Masa	m	kg
Velocidad	v	m/s
Aceleración	a	m/s ²
Fuerza	F	N
Presión	p o q	Pa
Densidad	ρ	kg/m ³

El Pascal es

$$1 \text{ Pa} = 1 \text{ N/m}^2$$

. Presión atmosférica y presión dinámica tienen la misma unidad, pero no son la misma magnitud.

2. Posición, velocidad y aceleración

La velocidad media entre dos instantes es:

$$\mathbf{v}_{\text{media}} = \frac{\Delta \mathbf{r}}{\Delta t}$$

La aceleración media es:

$$\mathbf{a}_{\text{media}} = \frac{\Delta \mathbf{v}}{\Delta t}$$

RocketPy integra ecuaciones de movimiento y proporciona muestras de la trayectoria. El proyecto consulta esas muestras; no reemplaza el integrador.

La rapidez es el tamaño de la velocidad:

$$V = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

3. Fuerzas y motor

La segunda ley resume la relación:

$$\sum \mathbf{F}_{\text{externas}} = m\mathbf{a}$$

En un cohete la masa cambia durante la combustión y el empuje no es constante. Por eso una cuenta de «empuje dividido por masa inicial» solo es una intuición. El motor local `Cesaroni_M1670.eng` y `RocketPy` resuelven el vuelo de referencia.

La gravedad, el empuje y el arrastre afectan la trayectoria. La simulación analizada empieza después de abandonar el riel y termina en el apogeo.

4. Velocidad respecto al aire

El viento hace que la velocidad del cohete y la del aire no sean iguales:

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{cohete}} - \mathbf{v}_{\text{aire}}, \quad V_{\text{rel}} = |\mathbf{v}_{\text{rel}}|$$

Flutter depende de lo que ve la aleta:

$$V_{\{\text{rel}\}}$$

, no simplemente de la velocidad respecto al suelo. En el caso base el viento es 4 m/s, pequeño frente a unos 286 m/s, pero no se descarta por definición.

5. Atmósfera y presión dinámica

La atmósfera local aporta presión estática

$$p$$

, densidad

$$\rho$$

y velocidad del sonido

$$a_s$$

. La presión dinámica es:

$$q = \frac{1}{2} \rho V_{\text{rel}}^2$$

Max-Q es el máximo de

$$q$$

en el intervalo elegido. No es una fuerza total, ni una presión uniforme sobre toda la superficie, ni un margen de flutter. Para convertirla en arrastre hacen falta coeficiente y área:

$$D = q C_D A_{\text{ref}}$$

En el caso base Max-Q es aproximadamente 41.60 kPa a 3.341 s. El valor de salida de las funciones atmosféricas cambia con la altitud.

6. Ángulo de ataque, Mach y estabilidad

El ángulo de ataque describe la orientación del vehículo respecto al flujo. Mach compara la rapidez relativa con el sonido:

$$\text{Ma} = \frac{V_{\text{rel}}}{a_s}$$

El centro de masa (CG) resume la distribución de masa; el centro de presión (CP) resume la acción aerodinámica del modelo. Una diferencia entre ambos, expresada en calibres, es el margen estático. Es un criterio de orientación, no una garantía de flutter.

7. Qué entrega RocketPy y qué añade este proyecto

RocketPy entrega vuelo,

$$V_{\{\text{rel}\}}$$

, altitud,

$$p$$

,

$$a_s$$

, Mach y estabilidad. `flutter.py` usa esas señales para calcular

$$V_f$$

. `study.py` calcula el ratio, el espesor requerido y resúmenes. La aleta no se deforma en la simulación de trayectoria.

8. Velocidad del sonido y por qué importa para el flutter

Para un gas ideal en la aproximación usada:

$$a_s = \sqrt{\gamma \frac{p}{\rho}}$$

Al ganar altitud disminuyen

$$p$$

y

$$\rho$$

, y la velocidad del sonido cambia de forma más suave que la presión. En la fórmula de Bennett, manteniendo la geometría:

$$V_f \propto a_s \sqrt{\frac{1}{p}}, \quad V_f \propto \frac{1}{\sqrt{\rho}}$$

Por eso

$$V_f$$

puede aumentar mientras el cohete asciende aunque la presión estática baje. El ratio final también depende de

$$V_{\{\text{rel}\}}$$

, así que no se debe interpretar la altitud aislada.

Siguiente paso: en el capítulo 02 se construye la frontera de flutter desde la geometría y el módulo de corte

$$G$$

.

02 · Flutter desde cero: aeroelasticidad de una aleta

Siguiente: programación

El proyecto es un *estudio de flutter de aletas para un cohete de alta potencia*. RocketPy es una caja negra de vuelo; este capítulo explica el cálculo empírico posterior. La fórmula procede de Bennett (2023) y no es una simulación modal de una estructura real.

1. Aeroelasticidad: por qué una aleta puede vibrar

Aeroelasticidad significa que el flujo y la deformación de una estructura se afectan mutuamente. Una aleta puede flexionarse, torcerse y cambiar el ángulo con el que encuentra el aire. Esa nueva orientación cambia la fuerza, y la fuerza puede alimentar la deformación.

El flutter aparece cuando el intercambio de energía del flujo con los movimientos supera la disipación efectiva. La palabra «aparece» aquí describe el fenómeno físico; el código solo calcula una frontera nominal de velocidad.

2. Vibración, resonancia y flutter no son sinónimos

- *Vibración*: oscilación alrededor de una posición.
- *Resonancia*: respuesta grande ante una excitación cercana a una frecuencia natural.
- *Flutter*: inestabilidad aeroelástica autoalimentada por el acoplamiento entre flujo y deformación.

Una vibración no implica flutter, y una cuenta de

$$V_f$$

no entrega frecuencias, amplitudes ni una película del movimiento.

3. Rigidez,

$$G$$

y torsión

En elasticidad lineal, una tensión tangencial

$$\tau$$

y una deformación angular

$$\gamma_s$$

se relacionan aproximadamente por:

$$\tau = G\gamma_s$$

El módulo de corte

$$G$$

tiene unidades Pa. El módulo de Young

$$E$$

describe otra deformación; no aparece en la fórmula de Bennett usada aquí.

$$G$$

es la única propiedad del material que entra en esa fórmula. La densidad se usa para la masa del vuelo, no para calcular

$$V_f$$

algebraicamente.

Las cifras de `materials.py` son valores típicos de referencia, no propiedades certificadas. Un fabricante, una orientación de fibras o una unión puede cambiar mucho

$$G$$

.

4. Geometría de la aleta

La aleta trapezoidal se describe con:

- c_r

: cuerda raíz.

- c_t

: cuerda punta.

- s

: barrido axial del borde delantero.

- b

: envergadura expuesta.

- t

: espesor constante.

- $S = (c_r + c_t) \frac{b}{2}$

: área.

- $AR = \frac{b^2}{S}$

: relación de aspecto.

- $\lambda = \frac{c_t}{c_r}$

: taper o estrechamiento.

El centroide axial que usa el código es:

$$C_x = \frac{c_r^2 + c_r c_t + c_t^2 + s(c_r + 2c_t)}{3(c_r + c_t)}, \quad \varepsilon = \frac{C_x}{c_r} - \frac{1}{4}$$

La aproximación exige

$$\varepsilon > 0$$

. Las propiedades `aspect_ratio` y `taper` de `Fin` alimentan directamente la fórmula.

5. Fórmula de Bennett (2023), término por término

La velocidad nominal de flutter implementada es:

$$V_f = a_s \sqrt{\frac{N_v}{D_v}}, \quad N_v = G(AR + 2) \left(\frac{\delta}{c_r} \right)^3, \quad D_v = \frac{24\varepsilon\gamma}{\pi} \cdot p \cdot AR^3 \cdot \frac{1 + \lambda}{2}, \quad \gamma = 1.4$$

Aquí:

Símbolo	Significado	Unidad
V_f	frontera nominal de flutter	m/s
a_s	velocidad local del sonido	m/s
G	módulo de corte de la aleta	Pa
p	presión estática	Pa
t, c_r	espesor y cuerda raíz	m
AR, λ, ε	relaciones geométricas	1
γ	razón de calores específicos	1

El cociente

$$\frac{t}{c_r}$$

no tiene unidad. Presión y

$$G$$

deben expresarse en las mismas unidades; el programa usa Pa.

La dependencia útil es:

$$V_f \propto \left(\frac{t}{c_r}\right)^{\frac{3}{2}}, \quad V_f \propto G^{\frac{1}{2}}, \quad V_f \propto p^{-\frac{1}{2}}$$

AR aparece en el numerador y en el denominador, por lo que no basta con decir «AR grande es mejor» sin evaluar toda la expresión. Bennett publica un ejemplo de aproximadamente 1425 ft/s; los valores exactos del chequeo local están en `test_flutter.py` y la prueba conserva esa referencia.

6. Ratio, margen 1.25 y Max-Q

El margen graficado es:

$$R_f(t) = \frac{V_f(t)}{V_{\text{rel}}(t)}$$

La demo exige

$$R_f \geq 1.25$$

. Es un límite de comparación, no una certificación. En la trayectoria base el mínimo es 2.0381962789. Ocurre cerca de Max-Q porque, bajo las hipótesis del modelo, la presión dinámica concentra la condición aerodinámica más exigente.

Para el caso base:

Métrica	Resultado
R_f en Max-Q	2.03819628
R_f en máxima velocidad	2.03884546

Son instantes y operaciones distintas. La diferencia pequeña no autoriza a redondearlos como si fueran la misma variable.

7. Espesor requerido

`required_thickness` invierte la fórmula para una velocidad objetivo

$$V_{\text{objetivo}} = R_{\text{objetivo}} V_{\text{rel}}$$

:

$$t_{\text{req}} = c_r \left(\left(\frac{V_{\text{objetivo}}}{a_s} \right)^2 \cdot \left(24\varepsilon \frac{\gamma}{\pi} \right) \cdot p \cdot \text{AR}^3 \cdot \frac{\frac{1+\lambda}{2}}{G \cdot (\text{AR} + 2)} \right)^{\frac{1}{3}}$$

El programa aplica la expresión a toda la historia y toma el máximo. Para la trayectoria base y

$$R_{\text{objetivo}} = 1.25$$

, el aluminio requiere 2.17 mm; los demás valores aparecen en README y `results.json`.

8. Barrido algebraico frente a resimulación

Un barrido de espesor o material reutiliza presión, sonido y rapidez de una trayectoria y recalcula

$$V_f$$

. Es postprocesado algebraico: *ignora el cambio de masa de la aleta*. Sirve para comparar rápido, no para sustituir una nueva trayectoria.

La aleta delgada de 1.2 mm sí se vuelve a simular. Su ratio re-simulado es 0.51033860; el cálculo algebraico da 0.51562740. La diferencia relativa guardada en `results.json` es 0.01036333, inferior al 2% documentado.

9. Escala uniforme y relación con Max-Q

Si todas las dimensiones del plano se multiplican por

$$k$$

, pero

$$t$$

no, AR,

$$\lambda$$

y

$$\varepsilon$$

permanecen iguales y

$$\frac{t}{c_r}$$

se divide por

$$k$$

. Por tanto:

$$V_f \rightarrow V_f k^{-\frac{3}{2}}$$

La aleta más grande no es automáticamente más segura frente a flutter. También cambia área y masa, y una resimulación debe estudiar esos efectos.

Con el factor geométrico

$$B$$

constante:

$$a_s^2 = \gamma \frac{p}{\rho}, \quad V_f^2 = a_s^2 \frac{GB}{p} = \frac{\gamma GB}{\rho}, \quad R_f^2 = \frac{\gamma GB}{\rho V_{\text{rel}}^2} = \frac{\gamma GB}{2q}$$

Esta es la *relación con Max-Q*: minimizar el ratio equivale a maximizar

$$q$$

bajo esas hipótesis. No es una ley universal de estructuras flexibles.

10. Qué no dice el modelo

El módulo modela aletas trapezoidales homogéneas, espesor constante y propiedades efectivas isotrópicas. No resuelve amplitud, rotura, modos, uniones, laminados, amortiguamiento ni divergencia. Rebasar la frontera señala riesgo según el modelo, no una rotura segura.

Siguiente paso: practica una evaluación de `Fin`, `flutter_speed` y `required_thickness` en el capítulo de programación.

03 · Programación desde cero: convertir una ecuación en un estudio

Siguiente: código y datos

No necesitas conocer Python para empezar. El objetivo es reconocer entradas, operaciones, decisiones y resultados.

1. Un programa y un módulo

Un programa es una secuencia de instrucciones. Un módulo `.py` guarda funciones y clases que otros archivos pueden importar. La ejecución de referencia la coordina `run_demo.py`; la física de la frontera está en `flutter.py`.

2. Variables y unidades

```
densidad = 1.0
velocidad = 100.0
presion_dinamica = 0.5 * densidad * velocidad**2
print(presion_dinamica)
```

La salida es `5000.0 Pa` porque las entradas se expresan en kg/m^3 y m/s . Python no añade unidades automáticamente.

`26e9` significa

$$26 \cdot 10^9$$

. En la configuración representa un módulo de corte en Pa.

3. Funciones y objetos

```
from flutter import Fin, flutter_speed

fin = Fin(0.20, 0.10, 0.12, 0.04, 0.003, 26e9)
vf = flutter_speed(fin, 80000.0, 330.0)
print(f"{vf:.1f} m/s")
```

`Fin` agrupa geometría y propiedad efectiva. `flutter_speed` recibe el objeto y el estado atmosférico. El resultado es una cifra; su interpretación exige revisar las hipótesis del capítulo 02.

4. Invertir una función

```
from flutter import Fin, required_thickness

fin = Fin(0.20, 0.10, 0.12, 0.04, 0.003, 26e9)
thickness = required_thickness(fin, 80000.0, 330.0, 1.25 * 250.0)
print(f"{float(thickness) * 1000:.2f} mm")
```

El espesor requerido toma una velocidad objetivo. El programa también acepta arrays de NumPy para recorrer toda una historia temporal.

5. Listas, bucles y barridos

```
from flutter import Fin, flutter_speed

fin = Fin(0.20, 0.10, 0.12, 0.04, 0.003, 26e9)
for pressure in [100000.0, 80000.0, 60000.0]:
    print(pressure, flutter_speed(fin, pressure, 330.0))
```

`study.py` repite una idea similar para diseños, vientos y espesores. Un barrido no inventa incertidumbre: solo evalúa las combinaciones que aparecen en la configuración.

6. Diccionarios y resultados

```
import json
```

```
data = json.loads(open("outputs/results.json", encoding="utf-8").read())
base = data["baseline"]
print(base["min_flutter_ratio"], base["required_thickness_mm"])
```

JSON conserva nombres junto con valores. `results.json` es más trazable que una cifra copiada de una gráfica porque incluye configuración y resúmenes.

7. Decisiones y pruebas

```
from materials import get_material
```

```
material = get_material("aluminio_6061")
if material["shear_pa"] > 20e9:
    print("módulo de referencia alto")
else:
    print("módulo de referencia bajo")
```

Las pruebas automáticas comprueban casos concretos: inversión del espesor, escalado uniforme, selección y propiedades de masa. Una prueba pasada verifica el código para las entradas elegidas; no convierte el modelo en una medición.

8. Flujo completo

```
demo.json
```

↓

```
Design + Environment + SolidMotor + Rocket + Flight
```

↓

```
presión, sonido, rapidez, altitud, Mach y estabilidad
```

↓

```
Fin + flutter_history + required_thickness
```

↓

```
resúmenes, barridos, JSON, CSV, PNG e HTML
```

Siguiente paso: consulta el mapa de archivos en código y datos, y luego aprende a leer las figuras.

05 · Las siete gráficas, explicadas desde sus ejes

Siguiente: metodología

Todas las imágenes proceden de outputs/ y usan la configuración congelada. El caso base es payload 0 kg, escala 1, espesor 3 mm y viento 4 m/s. El intervalo va de la salida del riel al apogeo.

Figura 01 · 01-flight.png

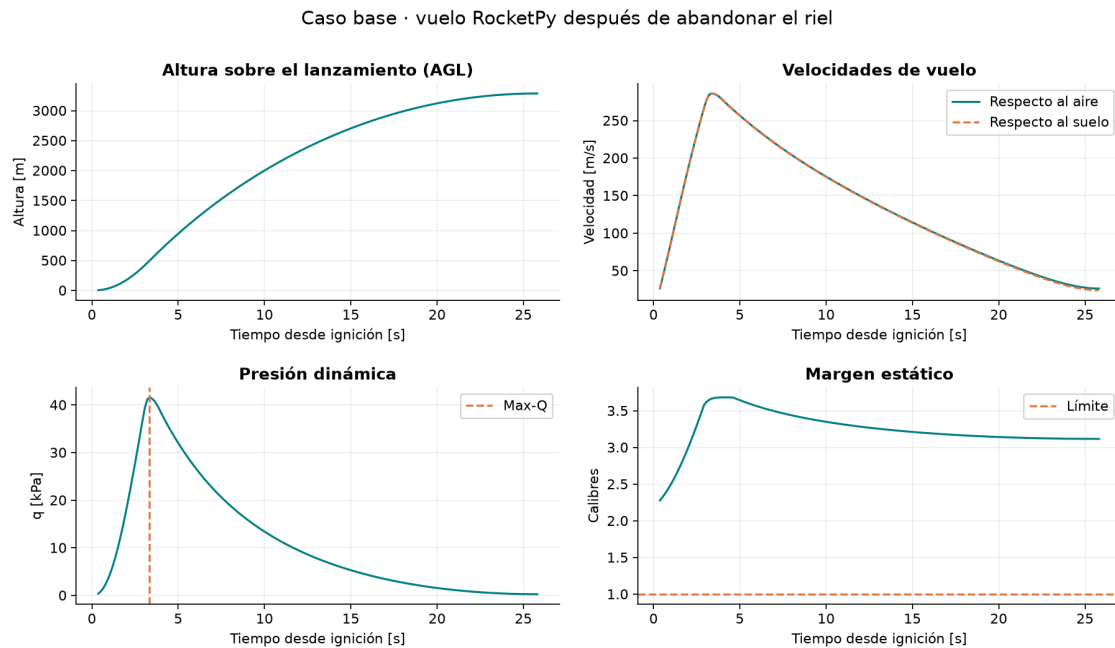


Figura 1: vuelo base

Es un resumen 2×2. Arriba izquierda: altura AGL frente al tiempo, que llega a 3287.34 m. Arriba derecha: rapidez respecto al aire y al suelo. Abajo izquierda:

q

en kPa y la línea de Max-Q, 41.60 kPa cerca de 3.341 s. Abajo derecha: margen estático en calibres y su límite.

Error común: leer AGL como ASL, o interpretar

q

como una propiedad de la aleta. Las curvas son muestras de un vuelo, no una cámara experimental.

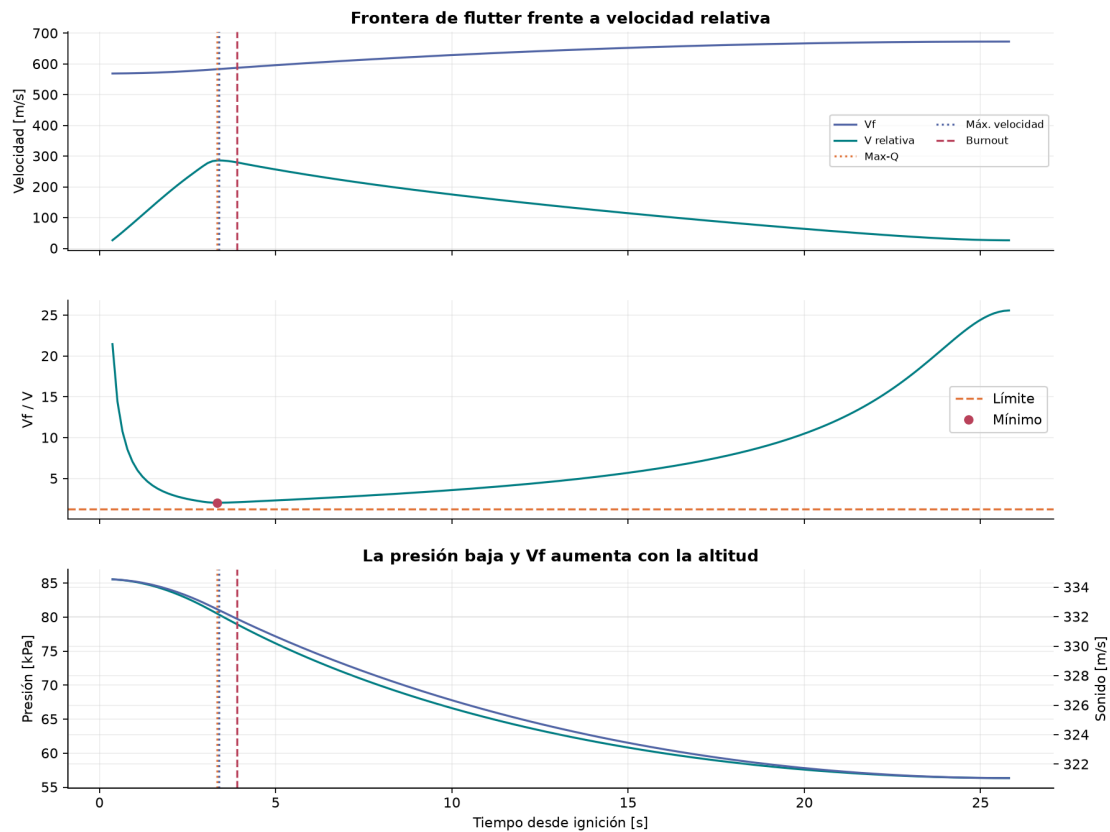
Figura 02 · 02-flutter-history.png

Figura 2: historia de flutter

Los tres paneles comparten tiempo. Arriba compara

$$V_f$$

con rapidez relativa y marca burnout, Max-Q y máxima velocidad. En el centro muestra

$$\frac{V_f}{V}$$

, su límite 1.25 y el mínimo. Abajo muestra presión estática y velocidad del sonido en ejes gemelos.

La lectura principal es que el ratio base alcanza 2.038 cerca de Max-Q. No hay que confundir

$$V_f$$

con la rapidez del vehículo.

Figura 03 · 03-altitude.png

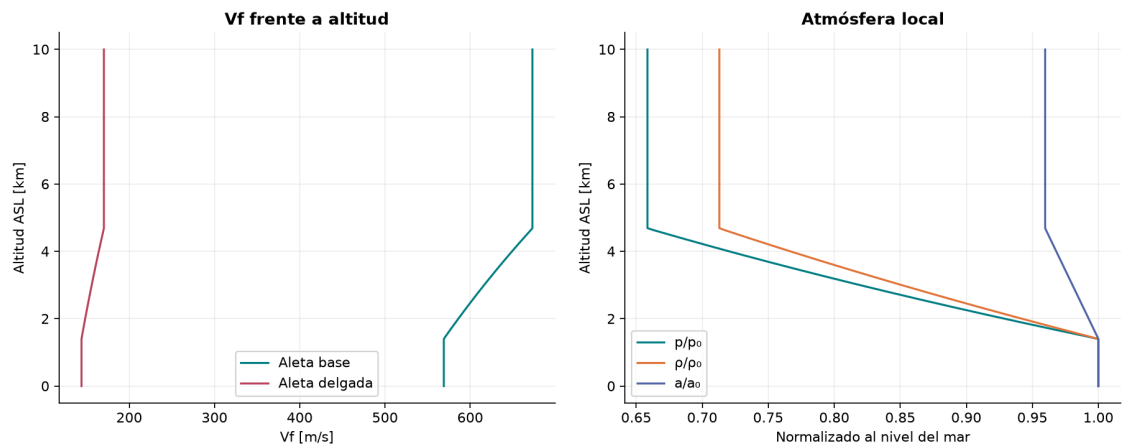


Figura 3: altitud y atmósfera

Izquierda:

$$V_f$$

frente a altitud ASL para aleta base y delgada. Derecha: presión, densidad y sonido normalizados al nivel de referencia.

$$V_f$$

puede aumentar con altitud porque la relación entre presión, densidad y sonido cambia; eso no significa que la rapidez de vuelo también aumente.

Figura 04 · 04-thickness-sweep.png

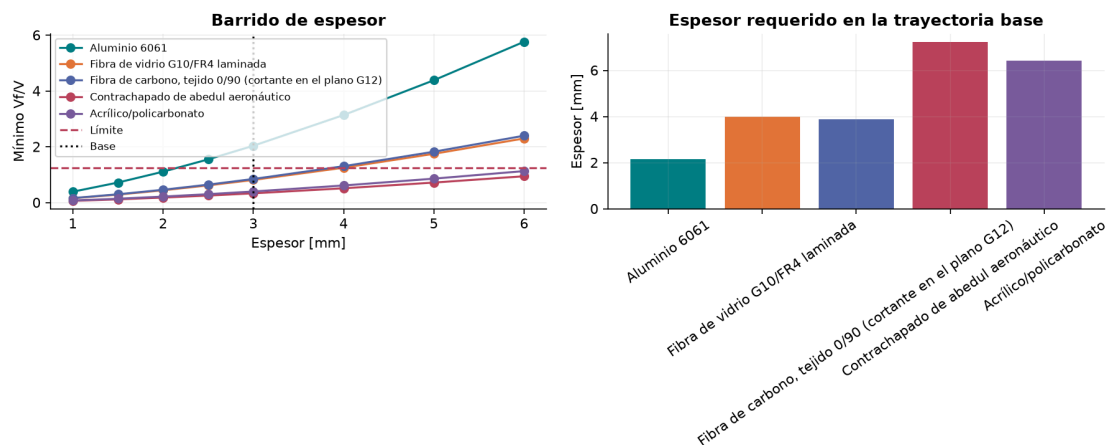


Figura 4: barrido de espesor

Izquierda: ratio mínimo frente a espesor, una curva por material, límite 1.25 y marcador de la base. Derecha: barras de espesor requerido para la trayectoria base. El eje horizontal de barras contiene nombres, no valores geométricos.

Como

$$V_f \propto t^{\{\frac{3}{2}\}}$$

, las curvas crecen con espesor. El barrido es algebraico y no cambia la masa del vuelo.

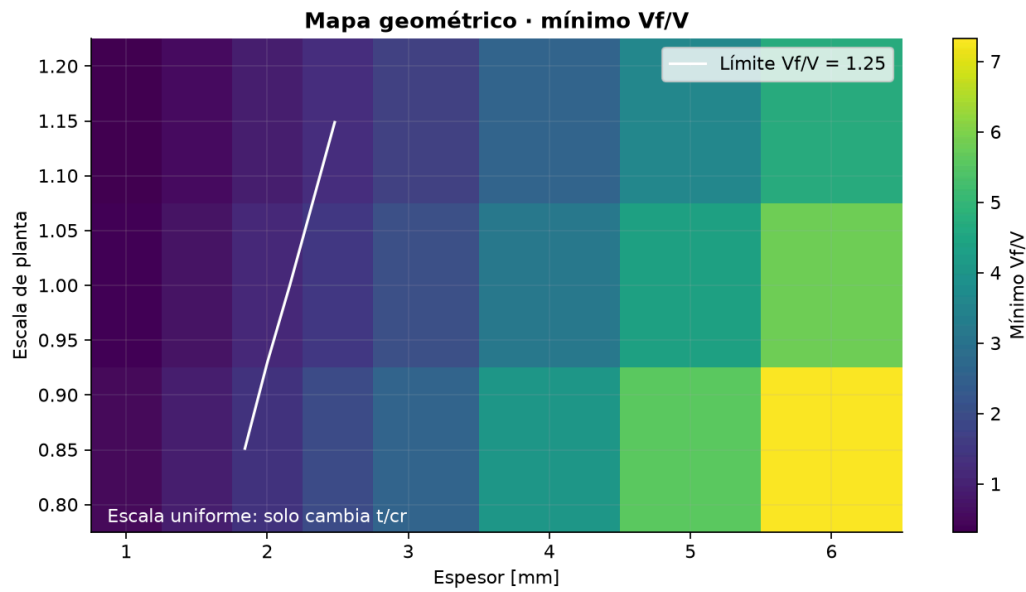
Figura 05 · 05-geometry-map.png

Figura 5: mapa geométrico

El mapa cruza escala de planta y espesor, usando el viento y payload base. El color es el ratio mínimo; la línea blanca es *Límite*

$$\frac{V_f}{V} = 1.25$$

. La escala uniforme cambia

$$\frac{t}{c_r}$$

, por eso aumenta o reduce el ratio aunque AR y taper permanezcan constantes.

No leer las celdas como simulaciones independientes de todos los diseños: el mapa usa postprocesado algebraico sobre las historias disponibles.

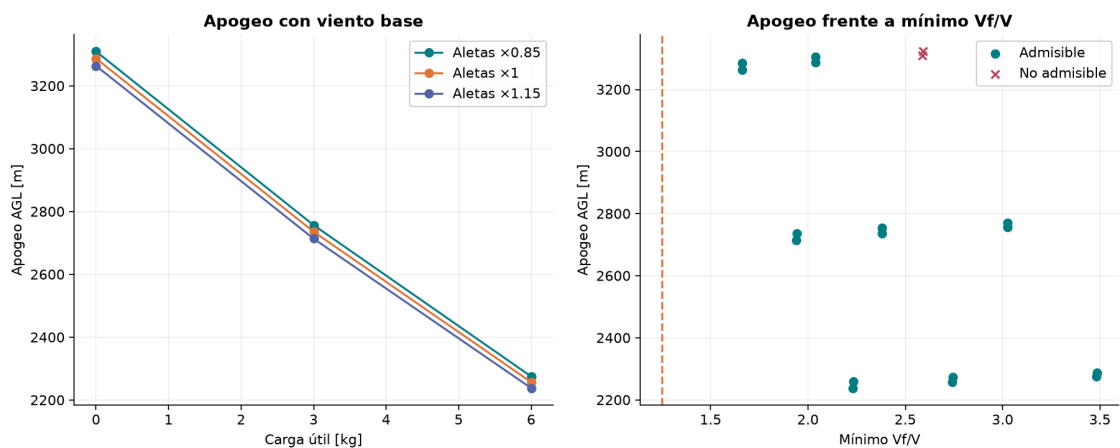
Figura 06 · 06-design-space.png

Figura 6: espacio de diseño

Izquierda: apogeo frente a payload para cada escala de aleta, con viento base. Derecha: apogeo frente a ratio mínimo; color y marcador indican admisible o no admisible. El punto seleccionado debe satisfacer todos los escenarios de viento previstos, no solo el punto más alto.

Figura 07 · 07-drag-sensitivity.png

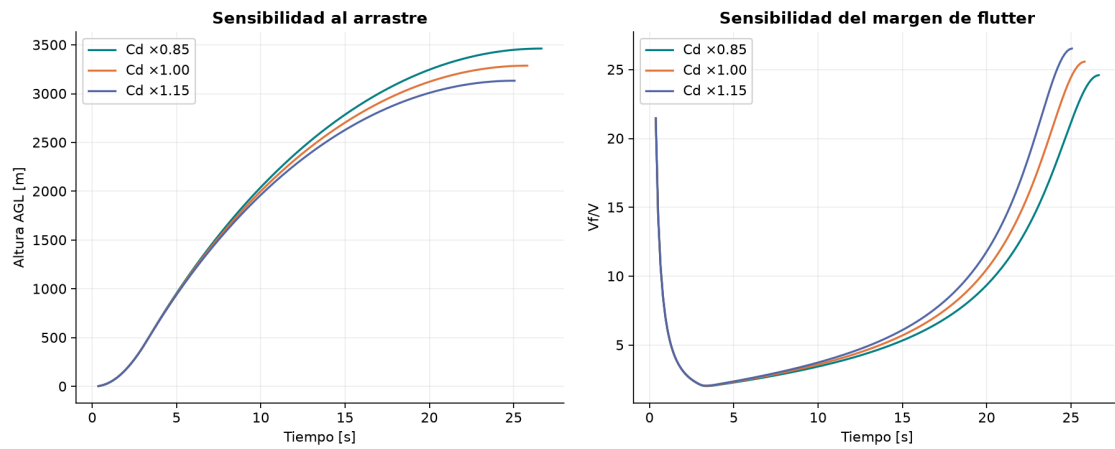


Figura 7: sensibilidad al arrastre

Izquierda: altitud frente al tiempo para

$$C_D$$

multiplicado por 0.85, 1 y 1.15. Derecha: ratio de flutter en esas tres trayectorias. Sirve para ver si la conclusión depende mucho del arrastre supuesto; no estima incertidumbre estadística.

Cómo escribir un pie de figura

Figura N. Qué se muestra, con unidades y caso utilizado. El patrón principal es . Esta lectura permite , pero no permite concluir . La fuente numérica es `archivo.csv` o la clave correspondiente de `results.json`.